Selective Patching of Agent Tool Knowledge under MCP Server Evolution

Tanmay student ID , teammate name and student ID

AI6101 Introduction to AI and AI Ethics, Group Project 3
Repository: https://github.com/NotTanJune/AI6101

Abstract

As MCP servers evolve, an agent harness can keep stale tool descriptions and usage rules. We compare three ways to update this knowledge on a 20-task subset of MCPEvol-Bench: no update, full rewrite, and selective patching. Our harness detects tool-, parameter-, and description-level changes, patches only the affected tool cards, validates the result, and promotes it only after a canary check. We report two experiments. In a historical pilot built on a Python implementation, selective patching had the highest Evolutionary Competency Score (ECS). In a live experiment, where the Rust harness ran end to end against the real evolved servers, full rewrite scored highest. In both experiments every paired difference between strategies falls within run-to-run noise, so neither experiment establishes a performance advantage. Selective patching did adapt at 41 to 65% of full rewrite's cost and left every unchanged tool card untouched. On tasks whose tools changed, it avoided failed calls caused by stale parameters and raised planning efficiency, although this gain is also not statistically significant.

1 Introduction

The Model Context Protocol (MCP) lets an agent discover a server's tools at run time (Model Context Protocol 2026). Many harnesses also keep curated tool knowledge, such as cleaned descriptions and usage rules, in a prompt or knowledge base (KB). This knowledge goes stale when a server adds, removes, or renames tools, changes parameters, or rewrites descriptions. The agent may then call a tool that no longer exists or send arguments that no longer match. Discovering the current tool list does not fix this if the harness still supplies the old descriptions and rules.

MCPEvol-Bench evolves real MCP servers through tool-, parameter-, and description-level mutations and shows that task fulfillment drops as servers change (Liu et al. 2026). MCP-Bench evaluates tool use against a fixed set of servers (Wang et al. 2025), so it cannot show whether a harness keeps up with change. We use MCPEvol-Bench's three metrics, Task Fulfillment (TF), Planning Effectiveness (PE), and ECS, and hold the model, decoding settings, and task protocol fixed across strategies.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Each strategy carries a different risk. **No update** costs nothing but leaves stale entries. **Full rewrite** regenerates the whole KB from the target definitions, so its cost grows with KB size and it can lose knowledge that was still correct. **Selective patch** updates only the fields the diff marks as changed, then validates the candidate and runs a canary before it replaces the prior KB.

With the base model, decoding settings and tasks held fixed, does selective patching of the Tool KB recover task performance on changed tools as well as a full rewrite, while retaining performance on unchanged tools and costing less to adapt?

This project contributes (i) a Tool KB format and a deterministic diff for tool, parameter, and description changes; (ii) patches limited to detected changes and checked before use; (iii) a canary gate kept separate from task-completion checks; (iv) a terminal agent for reviewing drift live and accepting or rejecting patches; (v) shared per-call cost and evidence logs; and (vi) an integration of the Rust harness with the teammate's Python agent and judge, tested against live MCP servers. Code, configurations, and run logs are in the project repository (AI6101 Group Project 3 team 2026).

2 Method and novelty

2.1 Tool KB and change detection

The Rust workspace has a core library for KBs, diffs, strategies, validation, canaries, and the cost ledger. An agent crate adds OpenAI and Gemini clients, an MCP client, and the agent loop, and a Ratatui crate provides the terminal interface. Headless benchmark runs and interactive sessions call the same core.

The Tool KB is a JSON document (schema `tool-kb/1`) with one card per tool. A card holds a stable `lineage_id`, server and tool identifiers, the tool's name and description, input and output JSON Schemas, usage rules, and provenance. Serialization sorts keys and cards, and the harness hashes each card with SHA-256. The agent sees tools only as the active KB describes them.

The diff matches source and target cards on `(server_id, lineage_id)` and compares the fields that affect behavior: `tool_id`, name, schemas, description, and usage rules. It ignores version labels, provenance, and the order of required fields. Each record classifies a change

as tool-, parameter-, or description-level and as an addition, removal, rename, or modification, with the field path and the old and new hashes. The diff never reads task text, so no strategy can pick relevant tools by looking at the benchmark tasks.

2.2 Adaptation and validation

No update returns the source KB without a model call. Full rewrite sends the target definitions and asks the model to write a description and usage rules for every tool without inventing new ones. Selective patch sends only the detected changes, with the source and target cards of the affected lineages. Both strategies send at most 12 cards per request, so request size does not grow with the KB. A patch consists of `set_field`, `remove_field`, `add_card`, and `remove_card` operations. Each operation must name a lineage and path that appear in the diff, and no card or field may be addressed twice. An empty diff makes no call.

For every card a strategy changes, the harness copies the identifiers, schemas, version, and provenance from the target, and the model writes only the description and usage rules. Validation already requires these structural fields to match the target exactly, so asking the model to copy large schemas only added rejections. If a response fails parsing or validation, the harness retries it up to twice with the rejection reason and then rejects the candidate. Because of this structural copy, replaying the stored patch alone does not reproduce the candidate. Validation checks that the candidate is schema-valid, matches the target's lineages, servers, tool IDs, names, and schemas, has no duplicate or removed tools, and contains every target card. For selective patch, every card outside the affected lineages must stay byte-identical to its source. A rejected candidate leaves the prior KB in place, and the harness logs the rejection.

2.3 Canary gate and limits

A candidate replaces the prior KB only after it passes a canary against the target version. The harness builds one to three targeted probes for changed tools, preferring parameter changes, then tool changes, then description changes, plus one control probe on an unchanged tool. Probe arguments come from schema constants, enumerations, defaults, or examples, and otherwise from a default value of the declared type. Tools whose names contain state-changing verbs are flagged as mutating.

The executor resets state before every probe and runs only when that state is isolated and disposable. Otherwise, or when no targeted probe can be built, the result is inconclusive. Promotion requires every probe to pass, and any failure or inconclusive result keeps the prior KB. The canary checks that the candidate matches the target. It does not check whether a user's task was completed.

The benchmark canary is simulated. It checks tool names and arguments against the target input schema (required fields, types, enumerations, and numeric bounds) without calling a live server. It catches stale names, missing required parameters, and type mismatches, but it cannot detect a misleading description, so a description-only change always passes. Live tool listings carry no lineage IDs. A tool whose name matches keeps its KB lineage, and any other tool gets `live:<name>`. A rename therefore appears as a removal plus an addition, and selective patch does not carry the old usage rules over to the new card. None of the supplied MCPEvol mutations record the old name of a renamed tool.

2.4 Live review and evidence

The terminal agent `mcpe` reads OpenAI and Gemini keys from environment variables and never stores or displays them. When it connects to a server, it turns the live `tools/list` into a temporary KB and diffs it against the session KB. The `/drift` view lists each change by server, tool, field path, class, and kind. The keys `n`, `f`, and `s` run no update, full rewrite, and selective patch. The view marks each probe as pass, fail, or inconclusive, and the user accepts a candidate with `a` or rejects it with `r`. The session KB changes only when the user accepts a candidate that passed the canary.

Every step writes a `run-event/1` JSONL record covering configuration, diff, outcome, probes, and model calls. Each call event records the phase, tokens, latency, rate card, estimated cost, and whether that cost is priced, unknown, mock, or local. Before a paid call, the ledger reserves a conservative estimate and refuses to dispatch the call if it would exceed the local cap. Unknown costs stay unknown rather than counting as zero, and the harness redacts secrets from all logs. The cap is a local limit, not the provider's bill.

2.5 Integration with the Python agent and judge

A process runner, `rust_runner.py` in `benchmark/integration/`, connects the Rust harness to the teammate's Python agent and judge. For each strategy, Rust builds the active Tool KB, launches the Python runner with a fixed argument contract, and validates every JSONL event the runner streams back against the shared `run-event/1` schema.

1MCP 0.38.3 serves the real MCPEvol server versions. We captured tool listings for original, iter3, and iter5 across the 23 servers that the 20 tasks use. The tasks also reference `fixparser-plugin-mcp`, which no configuration includes, as in the historical run. The live original listing matches the teammate's frozen snapshot byte for byte. Relative to original, iter3 leaves 252 of 287 tools unchanged and iter5 leaves 240. The rest were removed, added, or given new descriptions or input schemas.

Connecting the live path exposed six defects, all now fixed. (1) The Python runner rejected Rust's arguments and exited with code 2, which Rust reads as budget exhausted, so a wiring failure looked like a budget stop. The runner now accepts Rust's argument order and exits with code 64 on usage errors. (2) 1MCP 0.39.x rewrites public tool names, which breaks tool routing, so we pinned 0.38.3. (3) The `mcp-tasks` server fails to install through `npx`, so it runs from the bundled copy. (4) Each strategy sent every card in a single request, which exceeded the output limit or the request timeout. Requests are now batched with retries, and the harness copies structure itself (Section 2.2). (5) The configuration rejected a run whose source and target versions

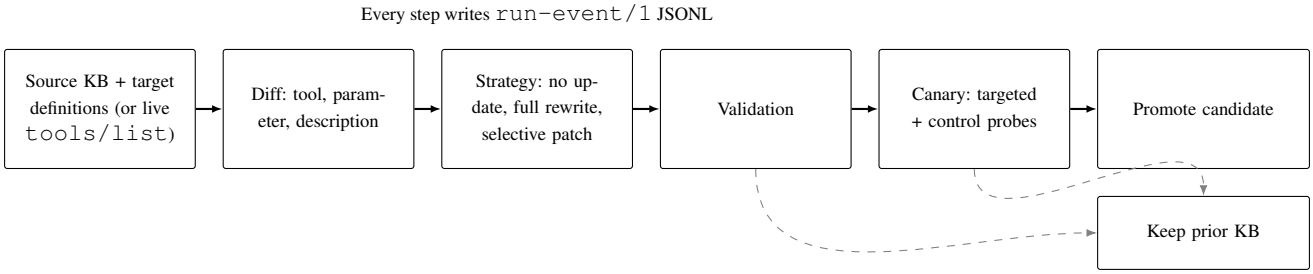

Figure 1: Adaptation pipeline shared by benchmark runs and the terminal agent.

were equal, which blocked the no-drift Early stage. An explicit `versions.baseline` option now allows it. (6) The Python KB check required one version label on every card, but selective patch keeps unchanged cards at their source version. The runner now accepts mixed versions.

3 Evaluation

The two experiments differ in implementation and must not be pooled. The *historical pilot* is the teammate's audited Python experiment. The *live experiment* reruns the same tasks through the Rust harness, whose strategies and KB generation differ from the Python ones.

3.1 Historical pilot

The pilot ran 20 tasks with one saved trajectory per task, strategy, and version (`original`, `iter3`, `iter5`), using gpt-6-luna as the agent. An LLM judge scored each trajectory five times on a 1 to 10 rubric covering task fulfillment, grounding, and planning and efficiency, with the order of dimensions shuffled. A task's ECS is its mean TF over the three versions minus their population standard deviation, and the reported ECS averages this over the 20 tasks. ECS is a score, not a pass rate. We compare strategies with a 10,000-sample paired bootstrap over tasks (seed 0).

The pilot has known limits. The judge prompt contradicts itself on redundant calls. Five judgments of one trajectory do not measure run-to-run variation. The judge sees tool outputs truncated to 500 characters. The selective probe validated schemas without calling tools.

3.2 Live experiment

The live experiment runs all 20 tasks with all three strategies at three stages, Early (original), Middle (iter3), and Late (iter5). Early has no drift and serves as a noise baseline. gpt-6-luna is the agent, judge, and adaptation model. We send no temperature, because gpt-6 models reject it, and fix the seed at 42. Each strategy and stage has one agent run per task with five judgments, and tasks run five at a time against one 1MCP endpoint. At each stage, 1MCP serves the matching server version and the KBs come from the captured listings. Early uses original as both source and target, and Middle and Late adapt from original to iter3 and iter5. Costs use the published gpt-6-luna standard rate card (accessed 2026-10-27), with a local cap of USD 20 per stage. All 180 task runs finished, and every event log passed the shared contract checker. Logs, KBs, trajectories, and judgments are in `benchmark/reports/live/`.

Table 1: Historical pilot (audited, 20 tasks): mean TF and PE per version (1 to 10) and ECS over the three versions.

	Original		Iter3		Iter5		
Strategy	TF	PE	TF	PE	TF	PE	ECS
No update	6.120	8.420	7.090	8.320	6.620	8.350	5.643
Selective patch	6.400	9.000	6.480	8.380	6.710	8.390	**5.740**
Full rewrite	6.490	8.390	5.650	9.020	6.230	8.390	5.306

Recovery and retention. A task is *changed* at a stage if any tool in its test case has a different name, description, or input schema from the original KB, and *unchanged* otherwise. Eight tasks are changed at Middle and nine at Late. Recovery compares strategies on changed tasks, and retention compares them on unchanged tasks, which should stay solvable. Early scores for the same tasks, where every strategy used the same KB, show how much scores vary by chance. The analysis script is in `benchmark/reports/live/analysis/`.

Completion verification. Neither experiment uses a separate state-based completion verifier. The judge's TF score, based on the final answer and the logged tool calls, is the only measure of task completion. The canary only decides whether a candidate KB is promoted, so a canary pass says nothing about task completion.

4 Results and discussion

4.1 Historical pilot

Table 1 gives the pilot scores and ECS, and Table 2 gives the paired differences for both experiments.

Selective patch has the highest pilot ECS and full rewrite the lowest, which matches the direction the hypothesis predicts. Every paired 95% interval includes zero, however, so the pilot cannot separate the strategies. TF per version is mixed, and the pilot does not show that selective patch recovers performance over no update.

4.2 Live experiment

Table 3 gives the live scores and Table 4 the adaptation cost.

Table 2: Paired task-level ECS differences over 20 tasks (10,000-sample paired bootstrap, seed 0). W/L/T counts tasks won, lost, and tied by the first strategy.

Comparison	Diff.	W/L/T	95% CI
Historical pilot			
Selective − no update	+0.096	10/7/3	[−0.735, 0.850]
Selective − full rewrite	+0.434	9/7/4	[−0.074, 1.031]
No update − full rewrite	+0.338	8/10/2	[−0.405, 1.188]
Live experiment			
Selective − no update	+0.028	11/7/2	[−0.362, 0.425]
Selective − full rewrite	−0.088	9/9/2	[−0.798, 0.632]
No update − full rewrite	−0.116	7/12/1	[−0.895, 0.611]

Table 3: Live experiment (Rust strategies; separate from the pilot): mean TF and PE per stage over 20 tasks, one agent run per task with five judgments, and ECS over the three stages.

	Early		Middle		Late		
Strategy	TF	PE	TF	PE	TF	PE	ECS
No update	6.660	8.340	6.850	7.900	7.130	7.670	6.093
Selective patch	6.540	8.670	6.520	8.160	7.020	8.390	6.121
Full rewrite	6.210	8.050	7.250	8.890	7.380	7.790	**6.208**

In the live run, full rewrite has the highest ECS, followed by selective patch and no update, and again every paired interval includes zero. The Early stage shows why. All three strategies used the same original KB there, yet their mean TF still differs by almost half a point, and two runs of the same task differ by 1.0 to 1.3 points on average. The strategy differences are smaller than this run-to-run variation. The live ranking also differs from the pilot's, which points to noise rather than a stable effect.

Tool-call errors point in the expected direction. With the stale KB, no update had about twice the rate of failed MCP calls at Middle and Late that it had at Early, while both adapted strategies stayed lower. The counts are small and come from single runs, so they do not establish an effect. Server state was also not reset between runs. Task 40, for example, appended notes to a shared file on every run, so later runs saw a changed workspace. We archived these files with the run evidence and restored the dataset afterwards.

4.3 Adaptation and canary cost

Adaptation cost comes from the adaptation-phase `model_call` events (Table 4). No update never calls the model. In the pilot, full rewrite makes no calls either, because the Python version copies the current server definitions. Live costs are rate-card estimates, not the provider's bill, and no live call had unknown usage.

Selective patch used 5 to 7 calls against 39 for full rewrite, at 41 to 65% of its cost and with less model time. Each selective stage needed one retry, and full rewrite needed none. Full rewrite also paid for a full pass at Early, where nothing had changed and the canary blocked its candidate. Every promoted candidate passed all of its probes. The no-drift candidates were inconclusive because there was no changed tool to probe. Adaptation accounted for 9% of the live experiment's total cost of USD 1.41, and task execution and judging made up the rest.

4.4 Recovery and retention

Table 6 splits the live scores by whether a task's tools changed. On changed tasks, TF is level across strategies. Most of these tasks are near the TF ceiling, and the agent usually finishes them even with stale knowledge by retrying after an error. The difference shows in PE instead. At Late, selective patch raises PE on changed tasks from 7.09 to 9.11 over no update, with a smaller gain at Middle. On the same tasks at Early, where the KBs were identical, the gap is small. This is the clearest recovery signal in the live data, but its 95% interval still includes zero.

On unchanged tasks, selective patch's cards are byte-identical to no update's, so any score difference between them is chance, and the patch caused no retention loss. Full rewrite, which rewrote every unchanged card, scored higher on unchanged tasks at Middle. The effect did not repeat at Late, and one significant interval among this many comparisons is weak evidence.

The pilot gives a weaker picture (Table 5). Its adapter flagged the same eight changed tasks at Iter3, and at Iter5 it flagged two more than the live diff. Selective patch recovered no TF on changed tasks at either version. Its effect on unchanged tasks was mixed, a small loss at Iter3 and a small gain at Iter5. Full rewrite lost TF on both changed and unchanged tasks at Iter3. Its cards differ on every task, so part of that loss may come from the rewrite, but one trajectory per task cannot separate this from chance.

4.5 Failure analysis

We classify failures using both score differences and trajectory evidence. A TF gap of at least 1.5 points between strategies on the same task marks a candidate. Agent and judge outputs are stochastic, so a score gap alone is not proof, and we call the examples below *behavioral candidates* unless the trajectory shows the cause.

Under-update. An under-update leaves a changed tool, parameter, description, or usage rule stale. The selective candidates updated every changed description, and validation forces every changed schema to match the target. The clearest under-update is therefore the no-update baseline, which misses every change. In task 29 at Late, the agent built a `run_process` call from the original card, which has no `mode` parameter. The iter5 server rejected it with *Invalid mode 'undefined'*. The agent failed once more before adding `mode` and succeeding. Its answer was correct, but PE fell sharply. With the patched card, the first call included `mode` and succeeded. Task 21 shows the same pattern.

A subtler under-update involves a usage rule written into a description. In task 229, the original `create_new_diagram` description told the agent to follow the guide that `start_session` returns. The iter5 description drops this, but the server still requires a session. Selective patch adopted the new description, so its card no longer

Table 4: Adaptation and canary cost per strategy. Pilot full rewrite copies the current server definitions and makes no model calls. Pilot selective time is wall-clock adaptation time and its dollar cost was not recorded. Live time is summed adaptation model latency; live cost is a rate-card estimate.

Experiment	Strategy	Stage	Calls	Input tokens	Output tokens	Time (s)	Cost (USD)	Canary (pass / fail / inconcl.)	Promoted
Pilot	No update	All	0	0	0	0.0	0.000	not applicable	n/a
Pilot	Full rewrite	All	0	0	0	not recorded	0.000	not applicable	n/a
Pilot	Selective patch	Iter3	11	4,444	671	18.8	not recorded	schema check only	n/a
Pilot	Selective patch	Iter5	19	7,616	1,433	49.5	not recorded	schema check only	n/a
Live	No update	All	0	0	0	0.0	0.000	not applicable	n/a
Live	Full rewrite	Early	40	110,679	42,646	410.0	0.032	0 / 0 / 1	no
Live	Full rewrite	Middle	39	108,563	40,640	387.4	0.031	4 / 0 / 0	yes
Live	Full rewrite	Late	39	113,947	43,902	461.0	0.033	4 / 0 / 0	yes
Live	Selective patch	Early	0	0	0	0.0	0.000	0 / 0 / 1	no
Live	Selective patch	Middle	5	52,102	15,150	169.5	0.013	4 / 0 / 0	yes
Live	Selective patch	Late	7	80,619	27,276	331.4	0.022	4 / 0 / 0	yes

Table 5: Recovery and retention in the historical pilot: mean TF on changed and unchanged tasks, from the teammate's pilot analysis.

Tasks (n)	Version	No update	Selective	Full rewrite
Changed (8)	Iter3	7.125	6.125	5.250
Unchanged (12)	Iter3	7.083	6.583	5.833
Changed (11)	Iter5	6.909	6.545	6.091
Unchanged (9)	Iter5	6.111	6.778	6.333

Table 6: Recovery (changed tasks) and retention (unchanged tasks) in the live experiment: mean TF / PE. Early rows give the same tasks with identical KBs.

Tasks (n)	Stage	No update	Selective	Full rewrite
Changed (8)	Early	7.28 / 8.05	7.95 / 8.72	6.85 / 8.03
	Middle	8.18 / 8.25	7.72 / 9.38	8.28 / 9.05
Changed (9)	Early	7.29 / 8.27	7.89 / 8.56	6.76 / 7.69
	Late	8.09 / 7.09	8.07 / 9.11	8.20 / 7.71
Unchanged (12)	Early	6.25 / 8.53	5.60 / 8.63	5.78 / 8.07
	Middle	5.97 / 7.67	5.72 / 7.35	6.57 / 8.78
Unchanged (11)	Early	6.15 / 8.40	5.44 / 8.76	5.76 / 8.35
	Late	6.35 / 8.15	6.16 / 7.80	6.71 / 7.85

mentions the requirement. Every `create_new_diagram` call, in every strategy and stage, failed with *No active session. Please call start_session first.* because the task's tool list does not include `start_session`. The patch did not cause this failure, but the field-level diff cannot see the rule. The pilot shows the same failure at Iter3. Selective patch updated the card's schema and description but left out the rule to call `start_session` first, so the agent called `create_new_diagram` directly and got the same error. Detecting cross-tool prerequisites and patching dependent cards together would address this.

Over-update. An over-update changes unrelated knowledge and breaks behavior that worked. Selective patch modified none of the unchanged cards, which validation enforces, while full rewrite rewrote all of them. In task 281 at Late, all five tools the task uses are unchanged. Full rewrite still rewrote their cards and, for example, turned the guidance in the `split_tasks` description into seven separate usage rules. Its agent then gave all six subtasks the same priority, and two of five judgments lowered TF for that reason. No update and selective patch, which kept the original cards, scored near the maximum. The other three judgments gave full marks and there is one run, so this remains a candidate. In the pilot, task 21 at Iter3 is a similar candidate. Full rewrite scored half of what the other two strategies scored, after making extra time queries, repeating `run_process` calls, and using the wrong working directory. Confirming it needs repeated runs, because in the live run full rewrite scored similarly low on task 21 at Early, where it used the original KB.

In both experiments, selective patch left the hash of every unaffected card unchanged, so we found no structural over-update in the selective method. Behavioral regressions remain possible.

4.6 Discussion

Selective patching helped most when a change breaks calls built from old knowledge. `run_process` gained required parameters and lost one, and the patched card made the first call valid. It made little difference when changes were compatible. Most changed tools only gained optional parameters or reworded their descriptions, so a stale card still produced valid calls, and only two of the nine changed tasks produced schema errors under no update. Selective patching also kept every unchanged card intact, while full rewrite spent more and rewrote knowledge that was still correct.

Selective patching failed or could not help in three situations. The server's own evolution dropped a precondition that lived only in description text (task 229). Some server faults lie outside the KB, such as `sequentialthinking` failing with *getThoughtHistory is not a function* at Late. Tool state also persisted across runs, so task 26 found a project that an earlier run had already created. Description-level changes were the hardest, because the diff sees that text changed but not what it means, and the simulated canary cannot test descriptions. Parameter-level changes were the easiest, because the harness copies schemas from the target and both validation and the canary check them. A tool-level

rename would lose its usage rules, but the original KB has no usage rules, so this case did not arise in the live run.

5 Conclusion

Neither experiment shows that selective patching improves task performance. Selective patching had the highest ECS in the pilot and full rewrite had it in the live run, but in both every paired difference falls within run-to-run noise, so the performance question remains open. Selective patching did meet the other two goals. It adapted at 41 to 65% of full rewrite's cost, kept every unchanged card byte-identical, and showed no retention loss. On changed tasks it kept TF level and raised PE by avoiding calls built from stale parameters, although that gain is not significant either. All results come from one agent run per task, which the brief allows when compute is limited. Stronger claims need three runs per strategy and more tasks whose changes break old calls.

The harness limits patches to detected changes, checks candidate structure and unchanged cards, and keeps the canary decision separate from task completion. In the terminal agent, the session KB changes only when the user accepts a candidate that passed the canary. These controls limit what adaptation can change, but they do not by themselves improve task outcomes. The integration work taught two practical lessons. An exit code with two meanings made a wiring failure look like a budget stop, and live runs need a pinned 1MCP version to route tools reproducibly.

The simulated canary cannot test description-only changes, so a pass shows schema compatibility only, and the LLM judge is the only measure of task completion. A state-based completion verifier and a reset of server state between runs are the main gaps to close. Selective patching kept unaffected knowledge intact, but it can still miss dependencies between tools that no schema records.

References

AI6101 Group Project 3 team. 2026. MCP tool-knowledge adaptation harness. https://github.com/NotTanJune/AI6101. Live-run evidence in `benchmark/reports/live/`; results commit to be recorded at submission.

Liu, H.; Hu, K.; Liao, J.; Wang, Q.; Qian, P.; Zhai, Y.; Feng, D.; Ding, B.; and Wang, H. 2026. MCPEvol-Bench: Benchmarking LLM Agent Performance Across Dynamic Evolutions of MCP Servers. arXiv:2607.14642. Code: https://github.com/Octobrist/MCPEvol-Bench, commit 888c779.

Model Context Protocol. 2026. Specification. https://modelcontextprotocol.io/specification. Accessed 2026-10-27.

Wang, Z.; et al. 2025. MCP-Bench: Benchmarking Tool-Using LLM Agents with Complex Real-World Tasks via MCP Servers. arXiv:2508.20453.